Quantum Optimization

Quick Recap

Earlier lessons introduced quantum gates, multi-qubit states, CNOT, entanglement, teleportation and IBM hardware. Day 13 explores how quantum circuits can participate in finding good solutions to difficult decision-making problems.

1. What Is Optimization?

Optimization means finding the best possible solution while respecting given conditions. Three ingredients define a problem:

• Candidate solutions: the available choices.

• Objective (or cost) function: a numerical score for each choice.

• Constraints: rules a valid choice must obey.

For minimization, write min C(x), where x is a candidate solution and C(x) its cost. For maximization, write max C(x).

Everyday analogy: Visiting five cities once each while minimizing total travel distance is an optimization problem. Finding a valid route is easier than proving it is the shortest.

2. Why Consider Quantum Optimization?

Routing, scheduling, network design and portfolio selection can have huge numbers of candidate solutions. Quantum circuits can represent and manipulate candidate states through superposition, interference and entanglement. Algorithms attempt to bias measurement toward high-quality choices.

Important limitation: This does not mean a quantum computer checks every solution and instantly finds the best one. Today, a general practical speed advantage over strong classical optimizers has not been established.

3. Hybrid Quantum–Classical Optimization

Classical computer	Quantum computer
Sets initial parameters and chooses updates	Prepares a parameterized circuit
Calculates the objective and checks progress	Executes gates and measures outcomes
Decides when to stop	Returns sampled measurement data

A parameterized circuit contains adjustable gate angles. A classical optimizer changes those angles based on the measured cost; the two systems repeat the process.

4. The Optimization Loop

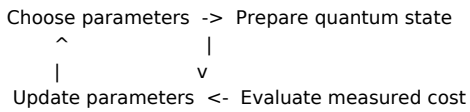


Figure 1. Repeated quantum evaluation and classical parameter updates.

5. Step-by-Step Workflow

1. Define the problem — State the objective and constraints, such as minimizing delivery distance.

2. Encode the choices — Convert the decision variables into a mathematical representation usable by a quantum circuit.

3. Prepare a parameterized circuit — Select a circuit structure and initial gate parameters.

4. Execute and measure — Run the circuit on a simulator or quantum processor and collect outcomes.

5. Evaluate cost — Compute how well the outcomes satisfy the objective.

6. Update parameters — A classical optimizer proposes new angles to improve the cost.

7. Repeat — Continue until a stopping condition is reached; convergence does not guarantee the global optimum.

6. Introducing QAOA

The Quantum Approximate Optimization Algorithm (QAOA) is a variational method for combinatorial optimization. It alternates two operations:

• Problem (cost) Hamiltonian: encodes which candidate solutions are better.

• Mixer Hamiltonian: helps move amplitude among candidate solutions.

A classical optimizer tunes the angles of these alternating operations. The number of alternating layers is often called reps or depth p.

The objective may be expressed as an expectation value: $E(\theta) = \langle\psi(\theta)|H|\psi(\theta)\rangle$, where H encodes the problem and θ denotes adjustable parameters.

7. Worked Example: Delivery Routes

Iteration	Route distance
1	420 km
2	390 km
3	372 km
4	360 km
5	358 km
6	358 km

This illustrative sequence shows improvement slowing over time. Equal scores at iterations 5 and 6 suggest a plateau, but do not prove the route is globally shortest. The workbook uses this example to explain convergence, not to report a measured quantum experiment.

8. Another Example: MaxCut

For a graph of four nodes connected in a cycle, MaxCut asks us to divide the nodes into two groups to maximize the number of edges crossing between groups. QAOA is often demonstrated on this problem because its objective can be encoded into a cost Hamiltonian.

Example edges: (0,1), (1,2), (2,3), (3,0). The alternating partition {0,2} versus {1,3} cuts all four edges, so its cut size is 4.

9. Qiskit Example: QAOA on MaxCut

The workbook introduces QAOA, COBYLA, StatevectorSampler, Maxcut and the Ising translation. The following reorganizes its example into a readable program; package compatibility depends on the installed Qiskit versions.

```
from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA
from qiskit.primitives import StatevectorSampler
from qiskit_optimization.applications import Maxcut
from qiskit_optimization.translators import to_ising
import networkx as nx

graph = nx.Graph()
graph.add_edges_from([(0,1), (1,2), (2,3), (3,0)])
maxcut = Maxcut(graph)
qp = maxcut.to_quadratic_program()
operator, offset = to_ising(qp)

qaoa = QAOA(sampler=StatevectorSampler(),
        optimizer=COBYLA(maxiter=100), reps=2)
result = qaoa.compute_minimum_eigenvalue(operator)
print(result)
```

What the components do: NetworkX defines the graph; Maxcut models the objective; to_ising creates a quantum operator; QAOA builds the alternating circuit; COBYLA tunes its parameters; the sampler provides circuit measurement information.

Output: The result contains an optimized energy estimate and related solver data, rather than just a 0 or 1. The energy must be interpreted using the model's Ising offset and objective definition to recover a MaxCut score.

Why results vary: Sampling randomness, optimizer initialization and hardware noise can change the outcome. A simulation may be more reproducible than noisy hardware.

10. Where Is Quantum Optimization Studied?

- Logistics: delivery routing, fleet allocation and warehouse scheduling.
- Finance: balancing portfolio return and risk.
- Manufacturing: machine and job scheduling.
- Telecommunications: network routing, bandwidth and resource allocation.
- Artificial intelligence: training and parameter-search problems.

These are research directions and candidate use cases, not evidence that quantum optimization currently outperforms classical methods in each industry.

Summary: What to Remember

- Optimization requires candidate choices, an objective function and constraints.
- Hybrid methods repeatedly combine quantum circuit measurements with classical parameter updates.
- QAOA alternates problem and mixer operations; its parameters are optimized classically.
- Convergence or a good sampled solution is not a guarantee of the global optimum.